

Experiment 2

AIM: Queries (along with sub-Queries) using ANY, ALL, IN, EXISTS, NOTEXISTS, UNION, INTERSET, Constraints. Example: - Select the roll number and name of the student who secured fourth rank in the class.

SOLUTION:

Query(Select): A query is command to retrieve data from a table

Syntax:

```
select column1, column2, ...  
from table_name  
where condition;
```

Sub Query:

A Query is written inside a query. That structure is called nested structure. Inside query is called as Sub Query. The result of inner query is used in execution of outer query. i. e, first inner query is executed after that, depending on inner query output outer query is executed.

Syntax:

```
select column1, column2, ...  
from table_name  
where column operator (  
    select column  
    from table_name  
    where condition  
);
```

Query: Write a query to create a dept table with constraints

```
SQL>create table dept (  
    deptno number primary key,  
    dname varchar2(20)  
);
```

OUTPUT:?

Query: write a query to insert data into dept

```
SQL>insert into dept values (10, 'IT');  
SQL>insert into dept values (20, 'CSE');  
SQL>insert into dept values (30, 'ECE');
```

OUTPUT:?

Query: write a query to display all data of dept table

```
SQL> select * from dept;
```

OUTPUT:?

Query: Write a query to create a student table with constraints.

```
SQL> create table student (  
    rollno number primary key,  
    name varchar2(20) not null,  
    marks number check (marks between 0 and 100),  
    deptno number,  
    foreign key (deptno) references dept(deptno)  
);
```

OUTPUT:?

Query: write a query to insert data into student table

```
SQL>insert into student values (1, 'ravi',95, 10);  
SQL>insert into student values (2, 'sita',88, 10);  
SQL>insert into student values (3, 'kiran',82, 20);  
SQL>insert into student values (4, 'anil', 75, 20);  
SQL>insert into student values (5, 'ram',65, 30);
```

```
SQL>insert into student values (6, 'john',55, 30);
SQL>insert into student values (7, 'tom',45, null);
SQL>insert into student values (8, 'lisa' 35, null);
```

Output:?

Query: write a query to display all data of student table

```
SQL>select * from student;
```

OUTPUT:?

Query: Write a query to display the list of names in the student table

```
SQL> SELECT name
      FROM student;
```

Output:?

Query: Write a query to display the name of the student whose roll number is 5.

```
SQL> SELECT name
      FROM student
     WHERE rollno = 5;
```

Output:?

Query: Write a query to display roll number and marks of the student whose name is 'sita'.

```
SQL> SELECT rollno, marks
      FROM student
     WHERE name = 'sita';
```

Output:?

Operators:

IN operator: It is used to find out the data from list of outputs present in inner query.

Syntax:

```
select column_name
from table_name
where column_name in (value1, value2, ...);
```

Query: Write a query to display roll number and name of students whose marks are 75 or 82 or 95.

```
SQL> select rollno, name
      from student
     where marks in (75, 82, 95);
```

OUTPUT:?

Syntax(with a subquery):

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (SELECT column_name FROM table_name WHERE condition);
```

Query: Write a query to display roll number and name of students who belong to any department.

```
SQL> SELECT rollno, name
      FROM student
     WHERE deptno IN (SELECT deptno FROM dept);
```

Output:?

ANY Operator: It is used to find out data from inner query, but the outer query satisfies any one of the data which is present in inner Query. We will use relational operators also like <, >, <=, >= etc.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY (SELECT column_name FROM table_name WHERE condition);
```

Query: Write a query to display roll number and name of students who scored more than any of the marks 45 or 55.

```
SQL> select rollno, name
```

from student
where marks > any (45, 55);

Output:?

Query: Write a query to display students who scored more marks than any student who scored below 50.

```
SQL> select rollno, name
from student
where marks > any (
  select marks
  from student
  where marks < 50
);
```

OUTPUT:?

ALL Operator: It used to find out data from inner query, but the outer query satisfies all of the data present in inner query.

Syntax: select column_name
from table_name
where column_name operator all (subquery);

Query: Write a query to display students who scored more marks than all students who scored below 90.

```
SQL> select rollno, name
from student
where marks > all (
  select marks
  from student
  where marks < 90
);
```

OUTPUT:?

EXISTS: It is co-related nested query – Here output returns depend on existed data within inner query.

Syntax:

```
select column_name
from table_name
where exists (subquery);
```

Query: Write a query to display roll number and name of students who belong to a department using EXISTS and department name is 'IT'.

```
SQL> select rollno, name
from student s
where exists (
  select *
  from dept d
  where s.deptno = d.deptno and d.name = 'IT'
);
```

OUTPUT:?

NOT EXISTS

Syntax:

```
select column_name
from table_name
where not exists (subquery);
```

Query: Write a query to display students who do not belong to any department.

```
SQL> rollno, name
from student s
where not exists (
```

```
select *  
from dept d  
where s.deptno = d.deptno  
);
```

OUTPUT:?

UNION: Union Operation is used to combine the result of two or more select queries.

Syntax:

```
select column_name from table1  
union  
select column_name from table2;
```

Query: Write a query to display students who scored above 80 or below 40.

```
SQL>select rollno from student where marks > 80  
union  
select rollno from student where marks < 40;
```

OUTPUT:?

INTERSECT: It is used to combine two select statements and returns the common rows from both the tables.

Syntax:

```
select column_name from table1  
intersect  
select column_name from table2;
```

Query: Write a query to display students who scored above 60 and below 90.

```
select rollno from student where marks > 60  
intersect  
select rollno from student where marks < 90;
```

OUTPUT:?

Query: Write a query to find the second highest marks in the student table

```
SQL> SELECT MAX(marks)  
FROM student  
WHERE marks < (SELECT MAX(marks) FROM student);
```

OUTPUT:?

Using RANK() function :

Query: Write a query to display 4th rank student

```
SQL> SELECT rollno, name, marks  
FROM (  
    SELECT rollno, name, marks,  
           RANK() OVER (ORDER BY marks DESC) r  
    FROM student  
)  
WHERE r = 4;
```

Output:?